# Sorting Algorithms

Muhammad Abdulkariim

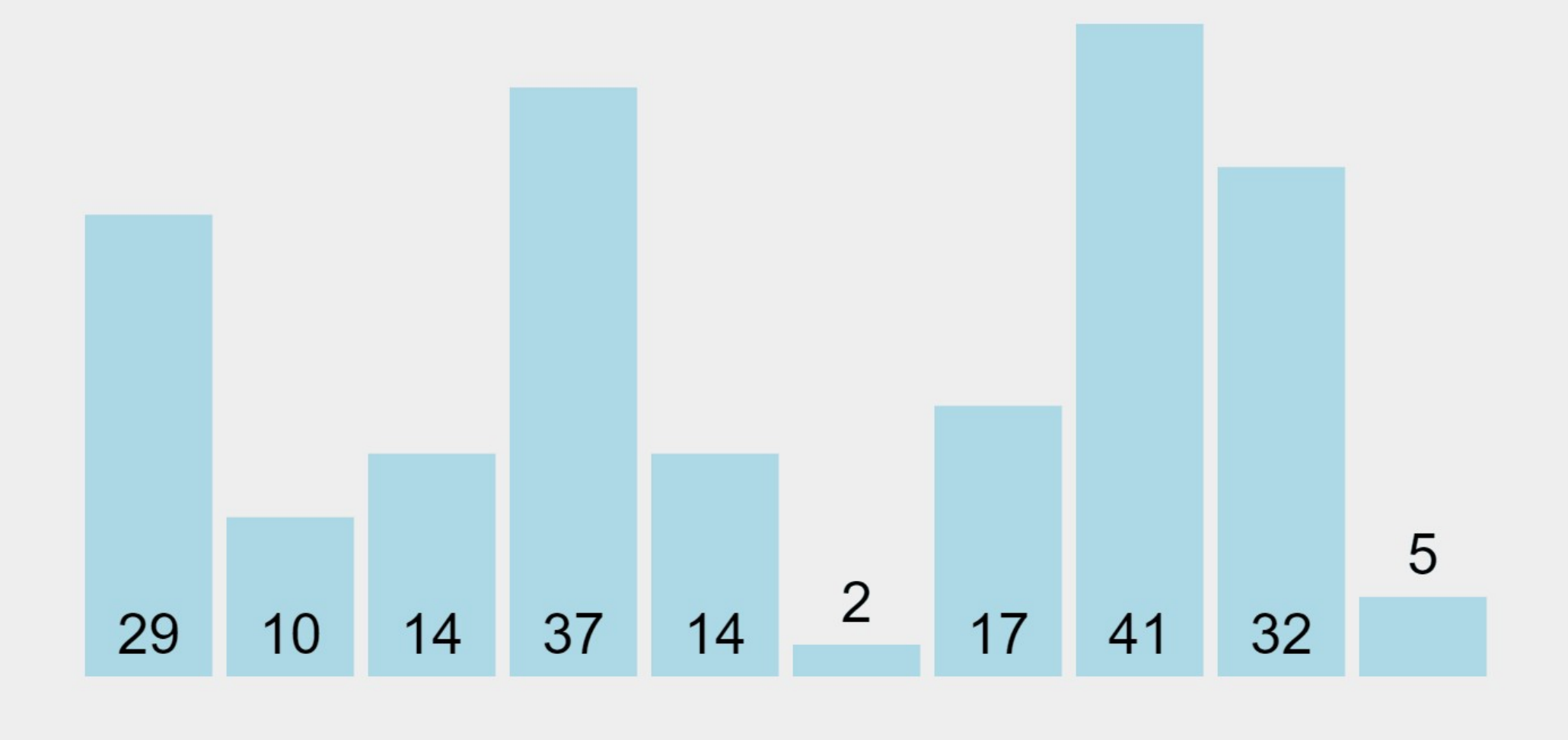
# Introduction to Sorting Algorithms

- **Definition:** Sorting algorithms are methods to rearrange elements in a list or array in a specific order (ascending or descending).
- **Importance:** Used in databases, search algorithms, and data processing.
- **Common Sorting Algorithms**: Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Quick Sort, etc.

# Bubble Sort

- Approach:
  - Compare adjacent elements and swap if they are in the wrong order.
  - Repeat until the list is sorted.
- Time Complexity:
  - Best Case: O(n) (already sorted)
  - Average Case: $O(n^2)$
  - Worst Case: $O(n^2)$
- Space Complexity: O(1) (in-place sorting)

# Bubble Sort



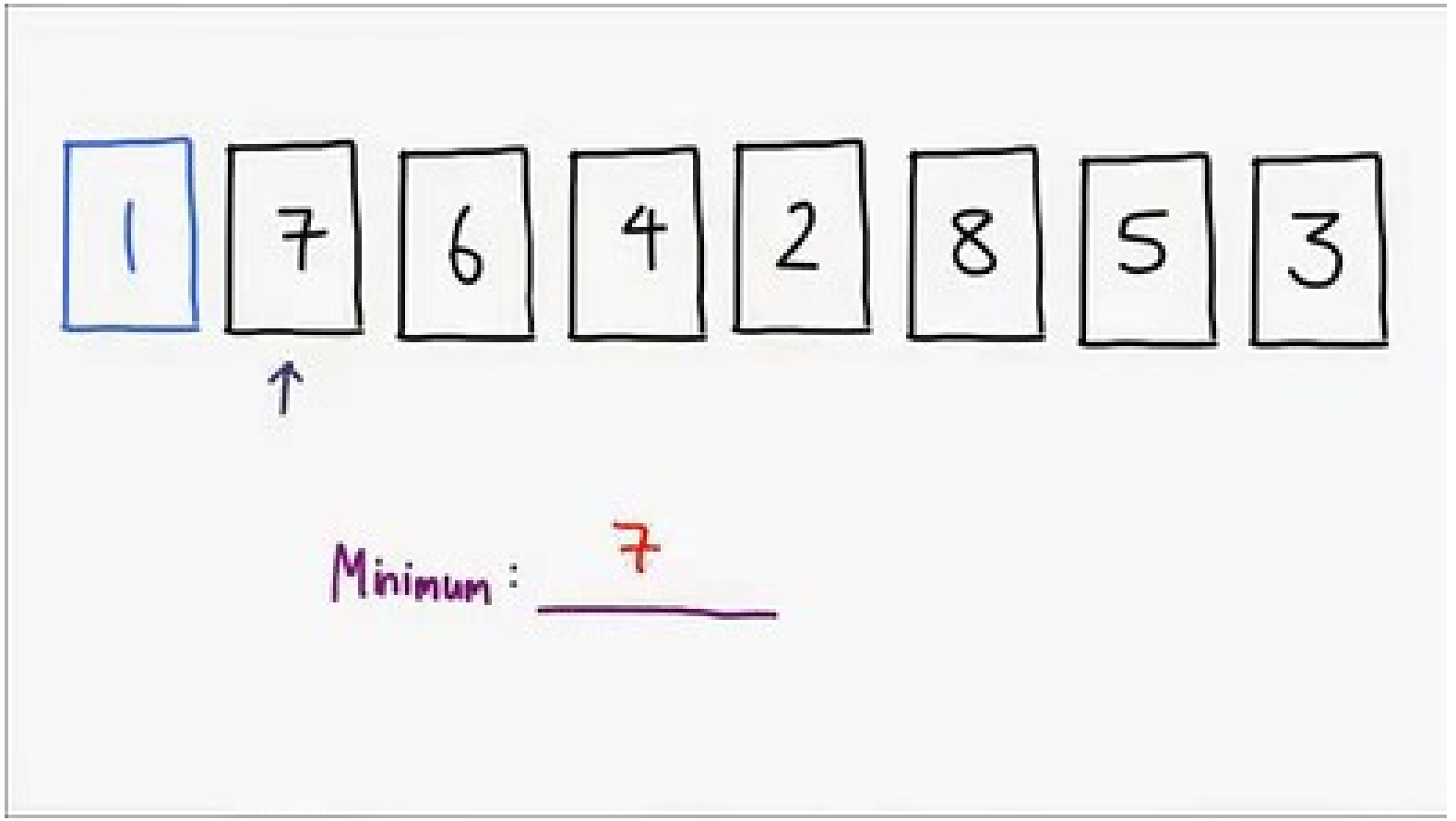
# Bubble Sort

```cpp
// An optimized version of Bubble Sort
void bubbleSort(vector<int>& arr) {
    int n = arr.size();
    bool swapped;

    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }

        // If no two elements were swapped, then break
        if (!swapped)
            break;
    }
}
```

# Selection Sort

- Approach:
  - Divide the list into sorted and unsorted parts.
  - Repeatedly select the smallest (or largest) element from the unsorted part and swap it with the first unsorted element.
- Time Complexity:
  - Best Case: $O(n^2)$
  - Average Case: $O(n^2)$
  - Worst Case: $O(n^2)$
- Space Complexity: O(1) (in-place sorting)

# Selection Sort



# Selection Sort

```
void selectionSort(vector<int> &arr) {
    int n = arr.size();

    for (int i = 0; i < n - 1; ++i) {

        // Assume the current position holds
        // the minimum element
        int min_idx = i;

        // Iterate through the unsorted portion
        // to find the actual minimum
        for (int j = i + 1; j < n; ++j) {
            if (arr[j] < arr[min_idx]) {

                // Update min_idx if a smaller
                // element is found
                min_idx = j;
            }
        }

        // Move minimum element to its
        // correct position
        swap(arr[i], arr[min_idx]);
    }
}
```

# Insertion Sort

- Approach:
  - Build the sorted list one element at a time.
  - Take each element and insert it into its correct position in the sorted part of the list.
- Time Complexity:
  - Best Case: O(n) (already sorted)
  - Average Case: $O(n^2)$
  - Worst Case: $O(n^2)$
- Space Complexity: O(1) (in-place sorting)

# Insertion Sort

# Insertion Sort

```
/* Function to sort array using insertion sort */
void insertionSort(int arr[], int n)
{
    for (int i = 1; i < n; ++i) {
        int key = arr[i];
        int j = i - 1;

        /* Move elements of arr[0..i-1], that are
           greater than key, to one position ahead
           of their current position */
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}
```

# Merge Sort

- Approach:
  - Divide the list into two halves.
  - Recursively sort each half.
  - Merge the two sorted halves.
- Time Complexity:
  - Best Case: O(n log n)
  - Average Case: O(n log n)
  - Worst Case: O(n log n)
- Space Complexity: O(n) (requires additional space for merging)

# Merge Sort

# Merge Sort

```cpp
// begin is for left index and end is right index
// of the sub-array of arr to be sorted
void mergeSort(vector<int>& arr, int left, int right)
{
    if (left >= right)
        return;

    int mid = left + (right - left) / 2;
    mergeSort(arr, left, mid);
    mergeSort(arr, mid + 1, right);
    merge(arr, left, mid, right);
}
```

```cpp
// Merges two subarrays of arr[]. First subarray is arr[left..mid]
// Second subarray is arr[mid+1..right]
void merge(vector<int>& arr, int left, int mid, int right){
    int n1 = mid - left + 1, n2 = right - mid;

    vector<int> L(n1), R(n2);

    // Copy data to temp vectors L[] and R[]
    for (int i = 0; i < n1; i++) L[i] = arr[left + i];
    for (int j = 0; j < n2; j++) R[j] = arr[mid + 1 + j];

    int i = 0, j = 0, k = left;

    // Merge the temp vectors back into arr[left..right]
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k] = L[i];
            i++;
        }
        else {
            arr[k] = R[j];
            j++;
        }
        k++;
    }

    // Copy the remaining elements of L[], if there are any
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

    // Copy the remaining elements of R[], if there are any
    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }
}
```

# Quick Sort

- Approach:
  - Choose a pivot element.
  - Partition the list into elements less than the pivot and elements greater than the pivot.
  - Recursively sort the partitions.
- Time Complexity:
  - Best Case: O(n log n)
  - Average Case: O(n log n)
  - Worst Case: $O(n^2)$ (if pivot selection is poor)
- Space Complexity: O(log n) (due to recursion stack)

# Quick Sort

6 5 3 1 8 7 2 4

# Quick Sort

```cpp
// The QuickSort function implementation
void quickSort(vector<int>& arr, int low, int high) {

    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // Recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

```cpp
int partition(vector<int>& arr, int low, int high) {

    // Choose the pivot
    int pivot = arr[high];

    // Index of smaller element and indicates
    // the right position of pivot found so far
    int i = low - 1;

    // Traverse arr[low..high] and move all smaller
    // elements on left side. Elements from low to
    // i are smaller after every iteration
    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    // Move pivot after smaller elements and
    // return its position
    swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

# Complexity Comaprison

| Algorithm | Best Case | Average Case | Worst Case | Space Complexity |
| --- | --- | --- | --- | --- |
| Bubble Sort | | | | |
| Selection Sort | | | | |
| Insertion Sort | | | | |
| Merge Sort | | | | |
| Quick Sort | | | | |

# Complexity Comaprison

| Algorithm | Best Case | Average Case | Worst Case | Space Complexity |
|---|---|---|---|---|
| Bubble Sort | O(n) | $O(n^2)$ | $O(n^2)$ | O(1) |
| Selection Sort | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ | O(1) |
| Insertion Sort | O(n) | $O(n^2)$ | $O(n^2)$ | O(1) |
| Merge Sort | O(n log n) | O(n log n) | O(n log n) | O(n) |
| Quick Sort | O(n log n) | O(n log n) | $O(n^2)$ | O(log n) |

# Quiz Time!

Q1) If an array is almost sorted, which algorithm is best to use? Why?

# Answer

**Insertion Sort** is optimal for almost sorted arrays.

**Why?** Its best case is O(n) when the array is nearly sorted because it only shifts out-of-place elements.

Other algorithms like Merge Sort or Quick Sort will still run in O(n log n).

Q2) In Quick Sort, what happens if you always pick the first element as pivot, and the array is already sorted?
What’s the time complexity?

# Answer

This is Quick Sort's **worst case.**

Each pivot leaves one empty partition and one of size n-1.

Becomes $O(n^2)$ time.

**Fix:** Use **randomized Quick Sort** or pick the middle/median element as pivot.

Q4) If an array is **reverse sorted**, which algorithms will hit their worst-case performance, and which might not be affected much?

# Answer

- **Worst case:**
  - **Insertion Sort:** $O(n^2)$
  - **Bubble Sort:** $O(n^2)$
  - **Quick Sort:** $O(n^2)$ if poor pivot choice (like first element)
- **Not much affected:**
  - **Merge Sort:** Always O(n log n)

Q5) Why is **Merge Sort** preferred over **Quick Sort** when sorting **linked lists**?

# Answer

- **Merge Sort** is preferred because:
  - Linked lists have **no random access**.
  - Merge Sort splits & merges in **O(1) extra space** using pointers.
- **Quick Sort needs random access** for partitioning—inefficient for linked lists.

# Kth Largest Element (LeetCode 215)

- Given an integer array nums and an integer k, return the **kth largest element** in the array. The kth largest element is the element that would be in position len(nums) - k if the array were sorted in **descending order**.
- **Example:**
  - **Input 1:** nums = [3,2,1,5,6,4], k = 2
  - **Output 1:** 5
  - **Input 2:** nums = [3,2,3,1,2,4,5,5,6], k = 4
  - **Output 2:** 4

# Approach 1: Full Sort

- Sort array in descending order.
- Return nums[k-1].
- Easy but **O(n log n)**.

# Approach 2: Min Heap

- Build a **min-heap of size k**. Push each element:
- If heap > k, pop smallest. The top of heap is the **kth largest**.
- Time: **O(n log k)**.
  Space: **O(k)**.

# Median of 2d Array

- Given a 2D array of integers, find its median value.
- Example:
  - Input: 1 6 5
    2 4 6
    7 22 9
  - Output: 6

# Median of 2d Array

**Solution**

1. Flatten the 2d array
2. Sort the array
3. Locate the median

**Complexity**

- **Time**: O((m*n)log(m*n))
- **Space**: O(m*n)

# Median of 2d Array II

- Given a 2D array of integers with sorted rows, find its median value.
- **Example**:
  - **Input**:
    1 3 5
    3 6 8
    2 6 9
  - **Output**: 6

# Median of 2d Array II – Approach 1

- **Solution**
  - Use Min-Heap
  - Start by inserting the first element of each row into the heap
  - Repeatedly extract the smallest element and push the next element from the same row
  - This continues until reaching the median position
- **Complexity**
  - **Time**: O(n*m*log(n)) since we are going to insert and remove the top element in priority queue (n × m) / 2 times
  - **Space**: O(n)

# Median of 2d Array II – Approach 2

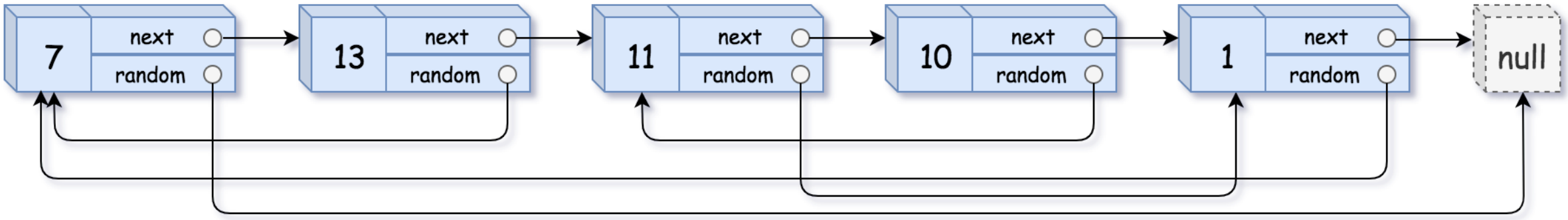
- **Solution**
  - Use binary search across the value range (from the minimum element to the maximum element in the matrix).
  - For each guess (mid value), count how many elements in the matrix are less than or equal to mid using binary search on each row.
  - Narrow the search range until finding the smallest value for which the count is at least half the total elements plus one (the median's position)
- **Complexity**
  - **Time**: O(n*log(m)*log(maxVal -minVal))
  - **Space**: O(1)

# Clone linked list with random pointer

Given a linked list where each node has two pointers:

- next: (points to the next node)
- random: (points to any node in the list or null)

Create an exact deep copy of the list (cloned nodes and exact same random and next structure).



# Clone linked list with random pointer – Approach 1

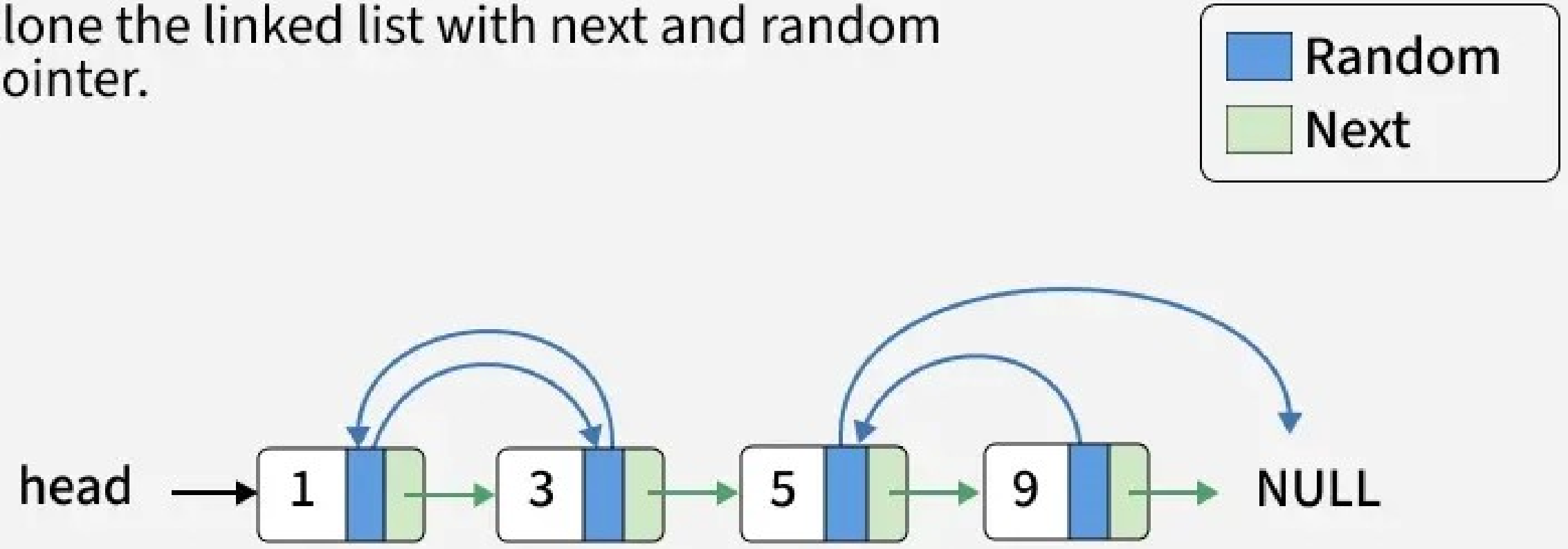
- **Solution**
  - Create a hash table, say **mp** to store the new nodes corresponding to their original nodes.
  - Traverse the original linked list and for every node, say **curr**, => Create a new node corresponding to curr and push them into a hash table, **mp[curr] = new Node()**.
  - Again traverse the original linked list to update the next and random pointer of each new node, **mp[curr]->next = mp[curr->next]** and **mp[curr]->random = mp[curr->random]**.
  - Return **mp[head]** as the head of the cloned linked list.
- **Complexity**:
  - **Time**: O(2n)
  - **Space:** O(n)

# Clone linked list with random pointer – Approach 2

- **Solution**
  - Iterate the original list and duplicate each node. The duplicate of each node follows its original immediately.
  - Iterate the new list and assign the random pointer for each duplicated node.
  - Restore the original list and extract the duplicated nodes.
- **Complexity**:
  - **Time**: O(3n)
  - **Space:** O(1)

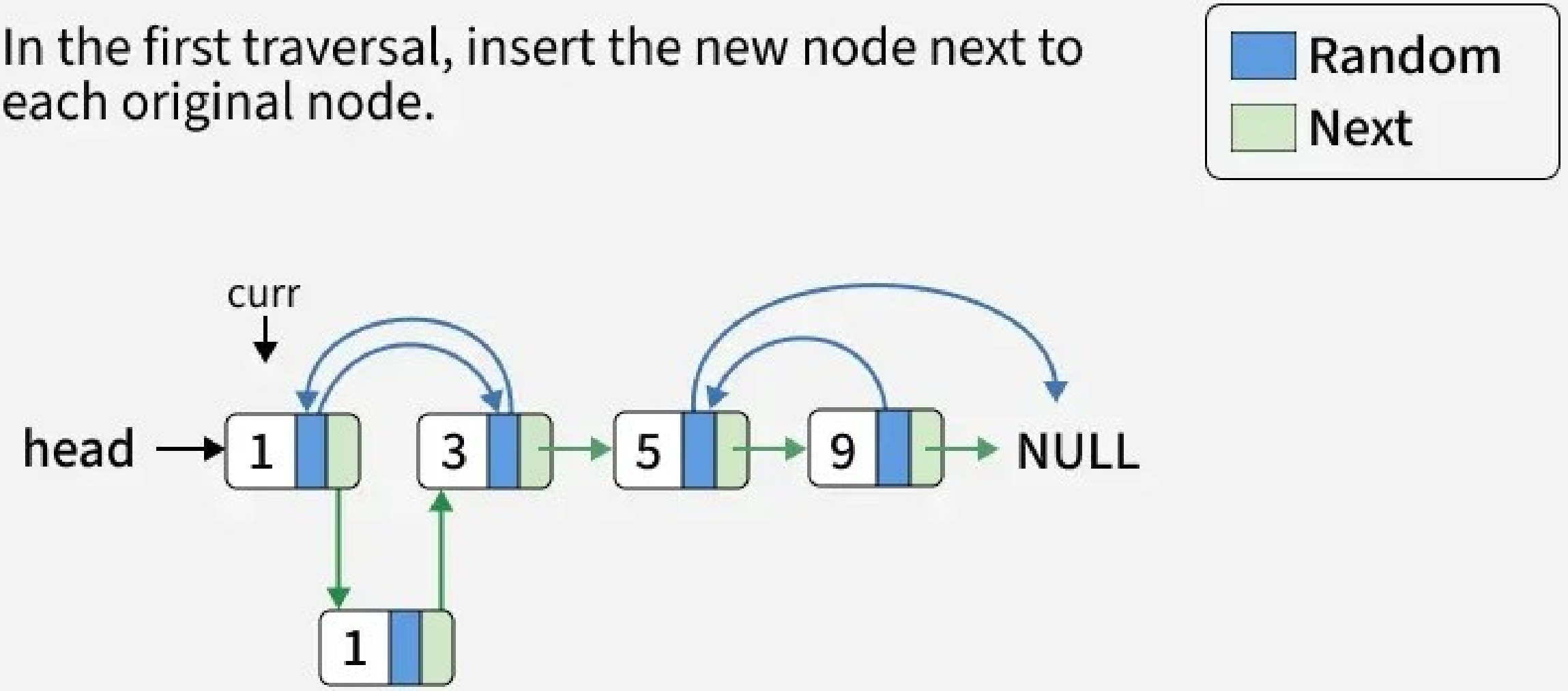
**01** Step

Clone the linked list with next and random pointer.



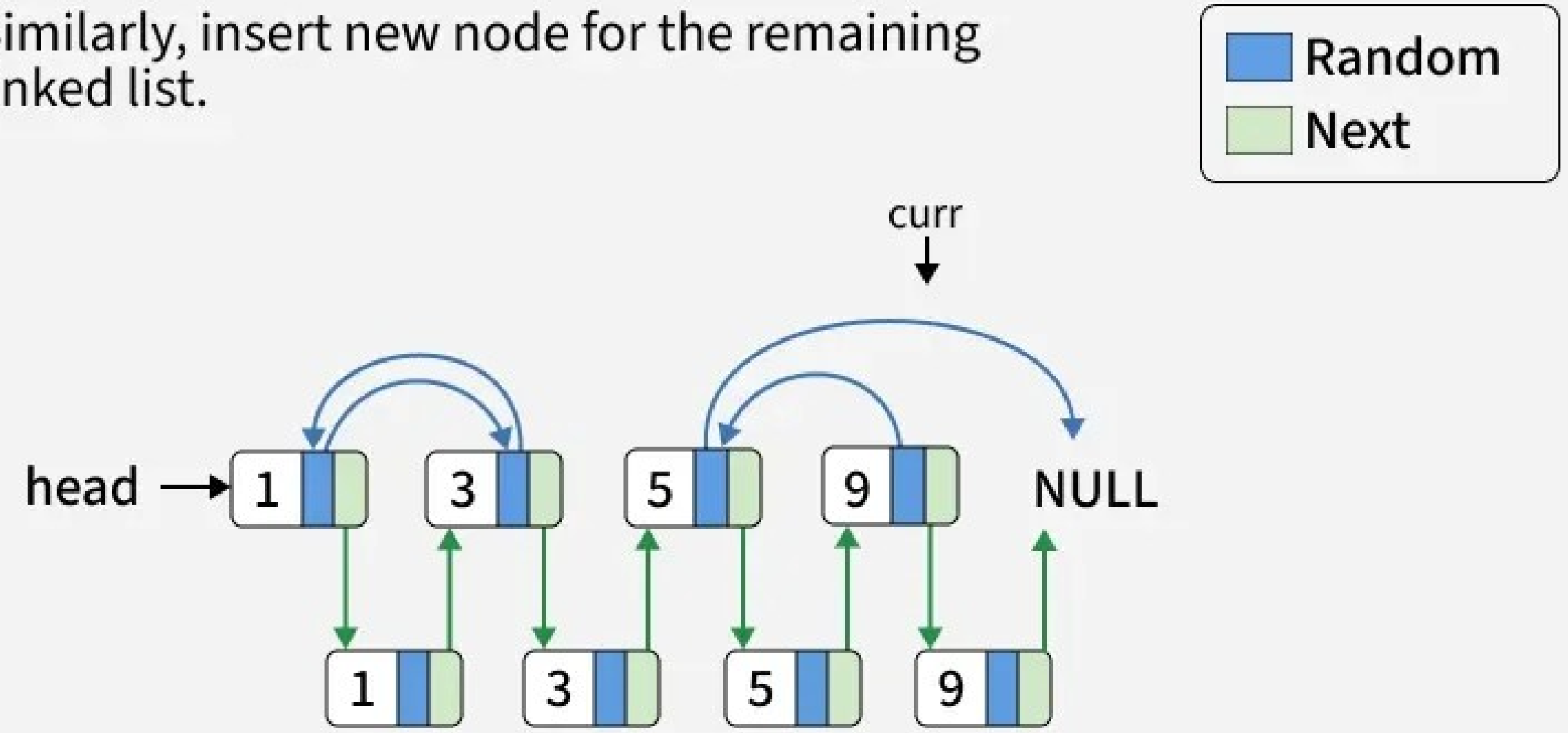
**02** Step

In the first traversal, insert the new node next to each original node.



**03** Step

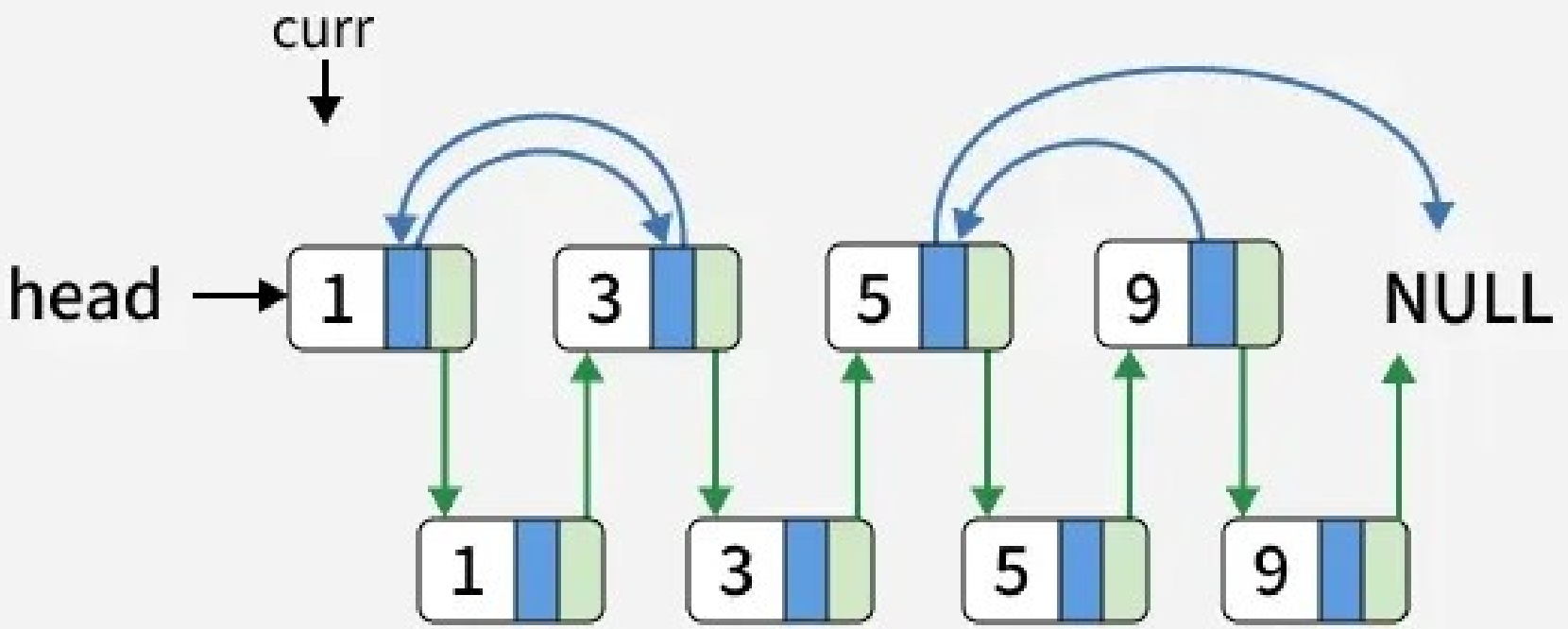
Similarly, insert new node for the remaining linked list.



**04**

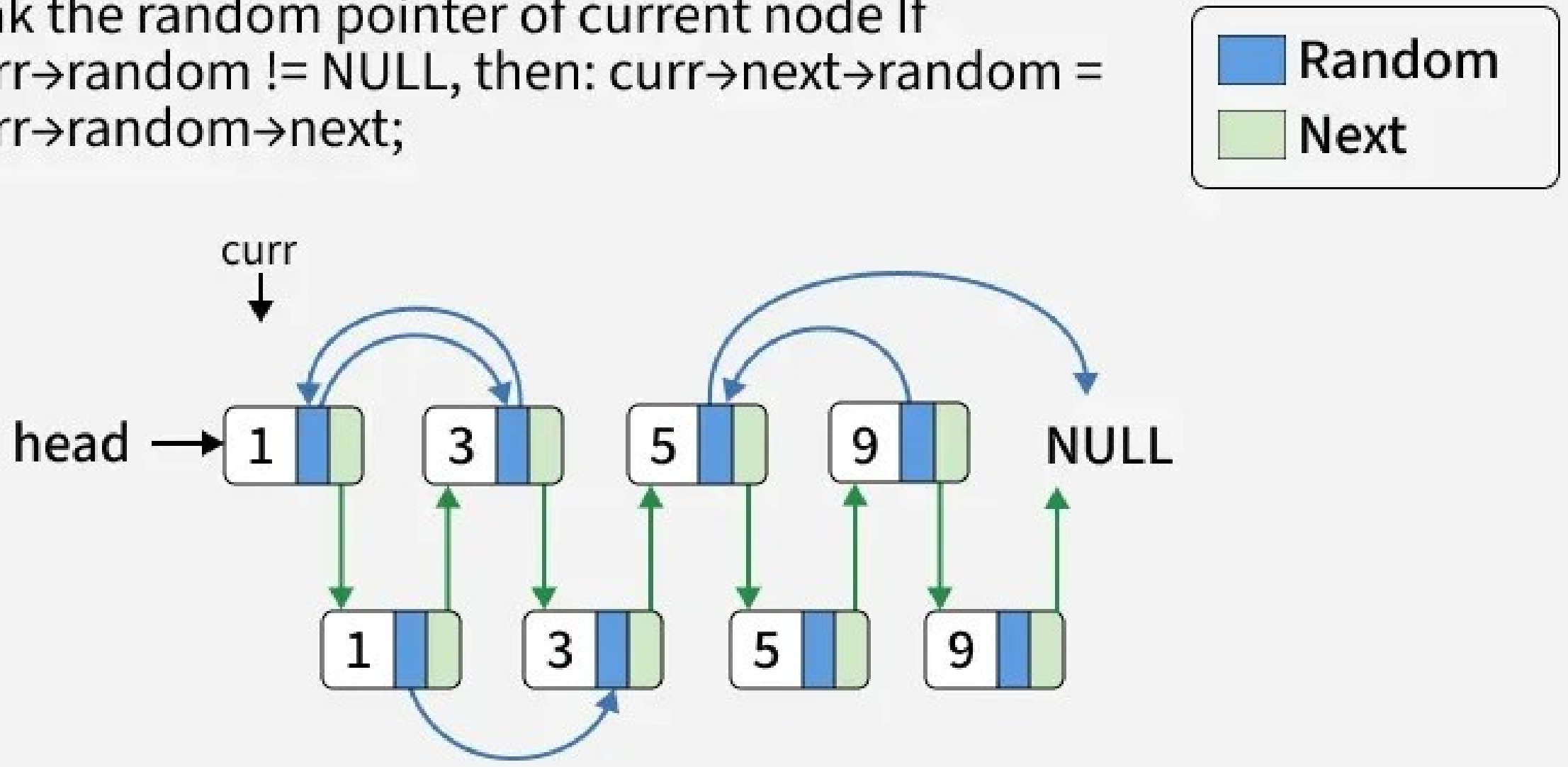
Step

In the second traversal, start connecting the random pointers of the cloned nodes and intailze the curr pointer to the head of orginal linked list.

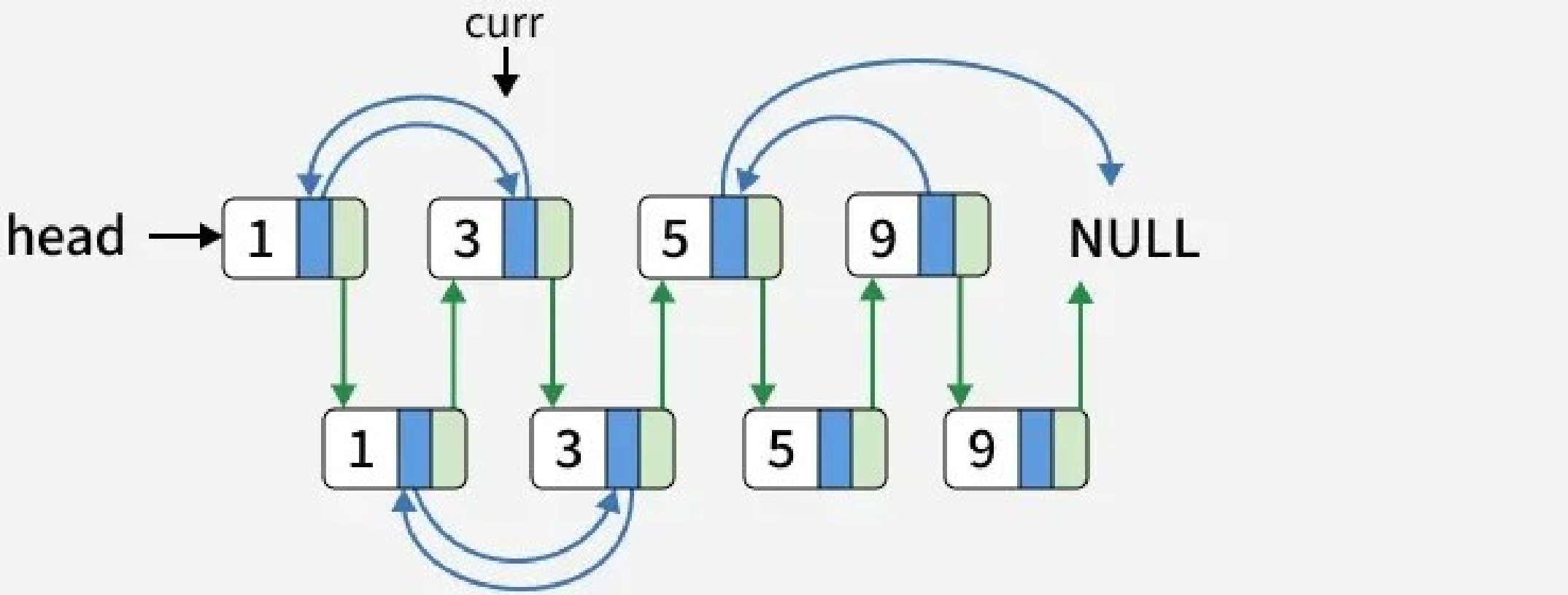
**05** Step

Link the random pointer of current node If curr→random != NULL, then: curr→next→random = curr→random→next;



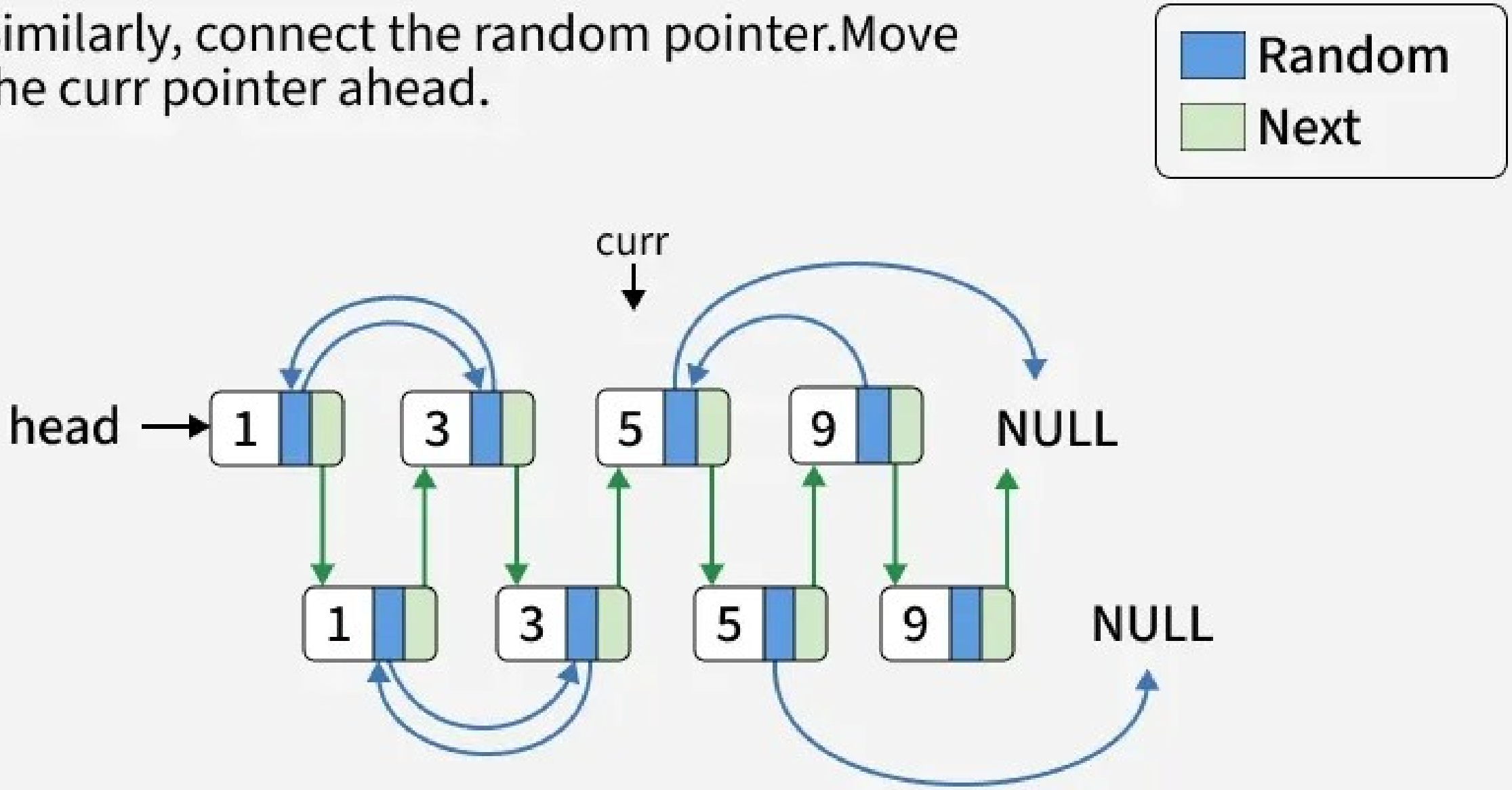
**06** Step

Similarly, connect the random pointer.Move the curr pointer ahead.



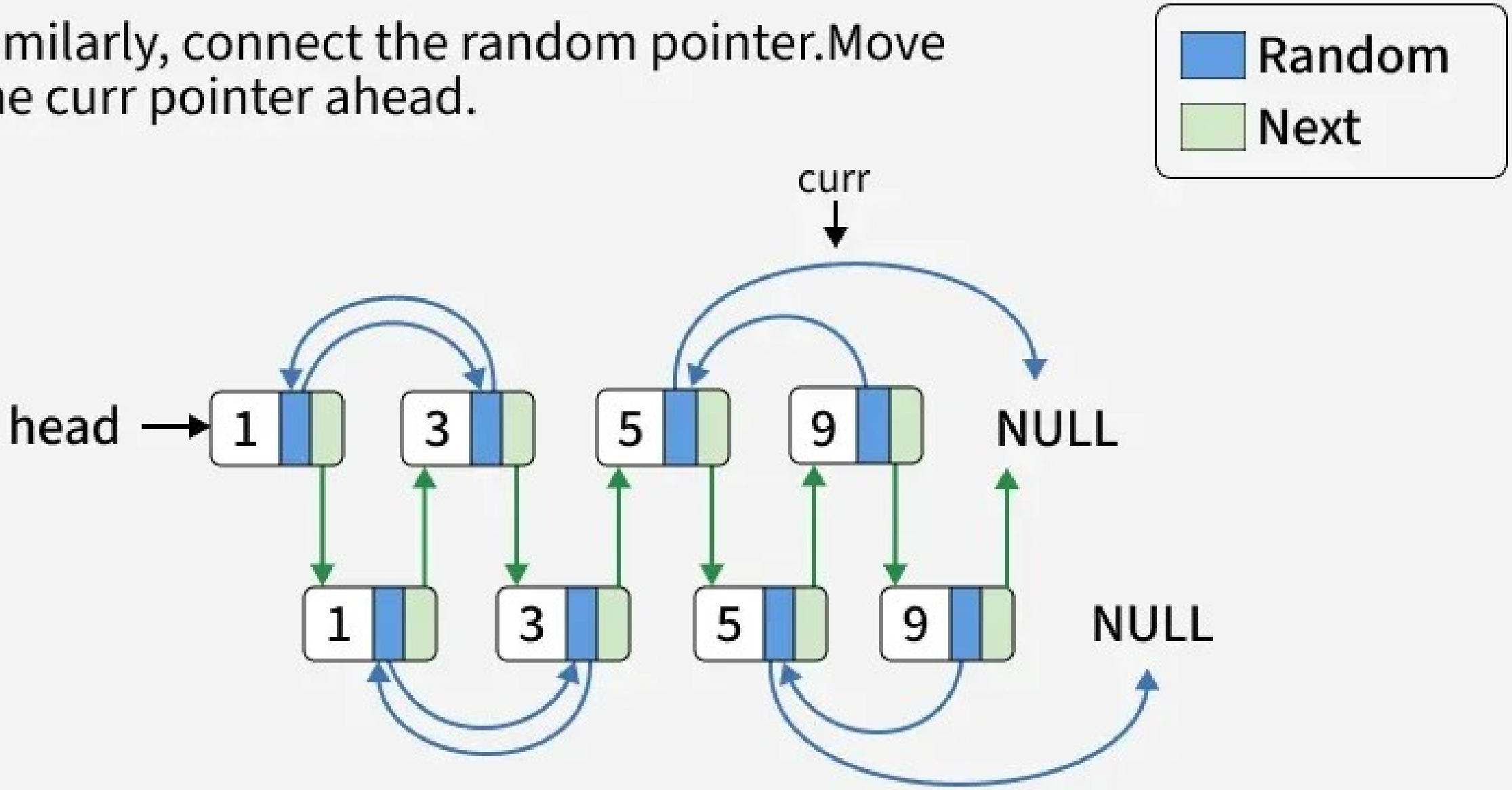
**07**
Step

Similarly, connect the random pointer.Move the curr pointer ahead.



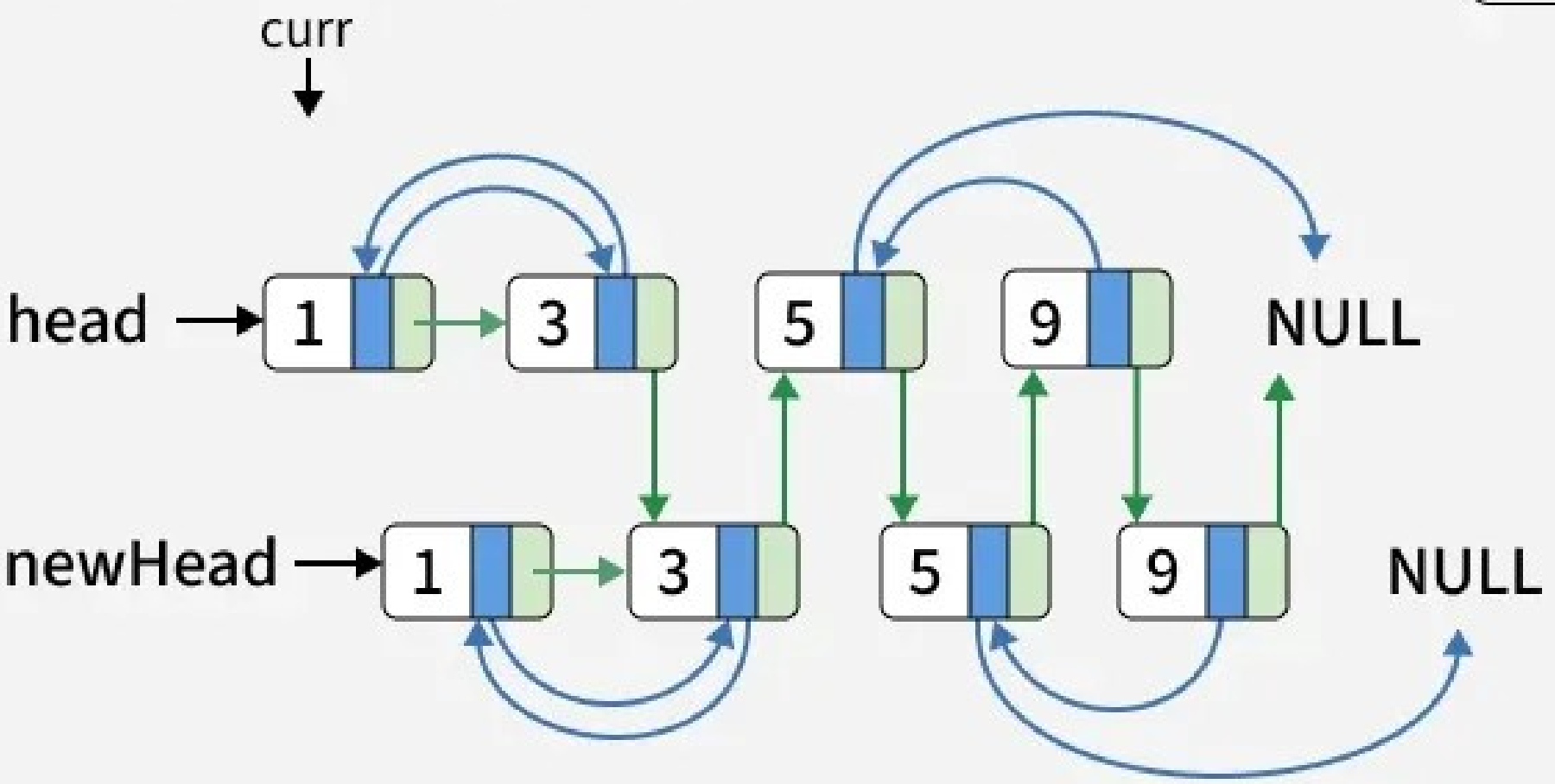
**08** Step

Similarly, connect the random pointer.Move the curr pointer ahead.



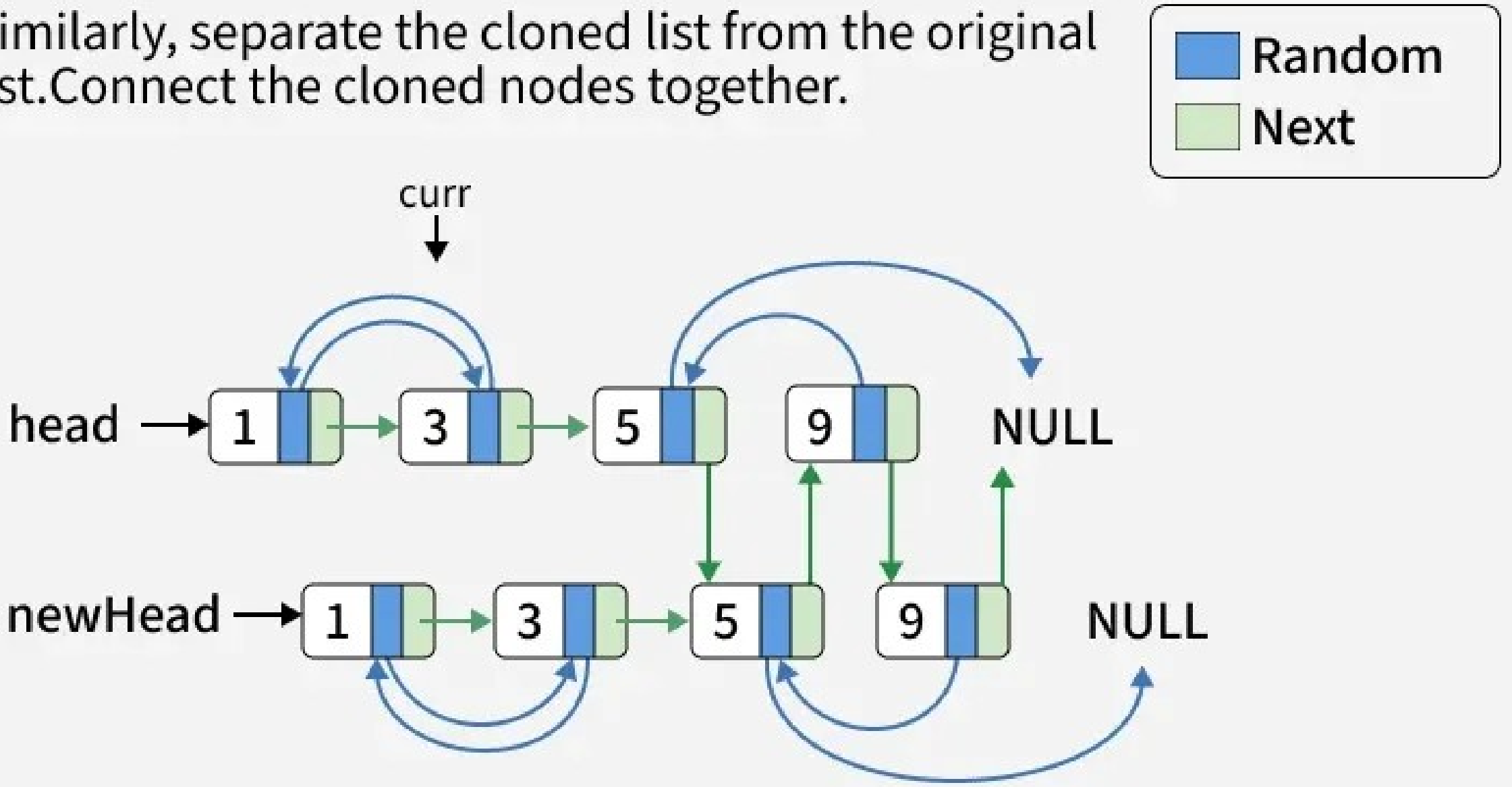
**09** Step

In the third traversal, separate the cloned list from the original list. Restore the original list.Connect the cloned nodes together.



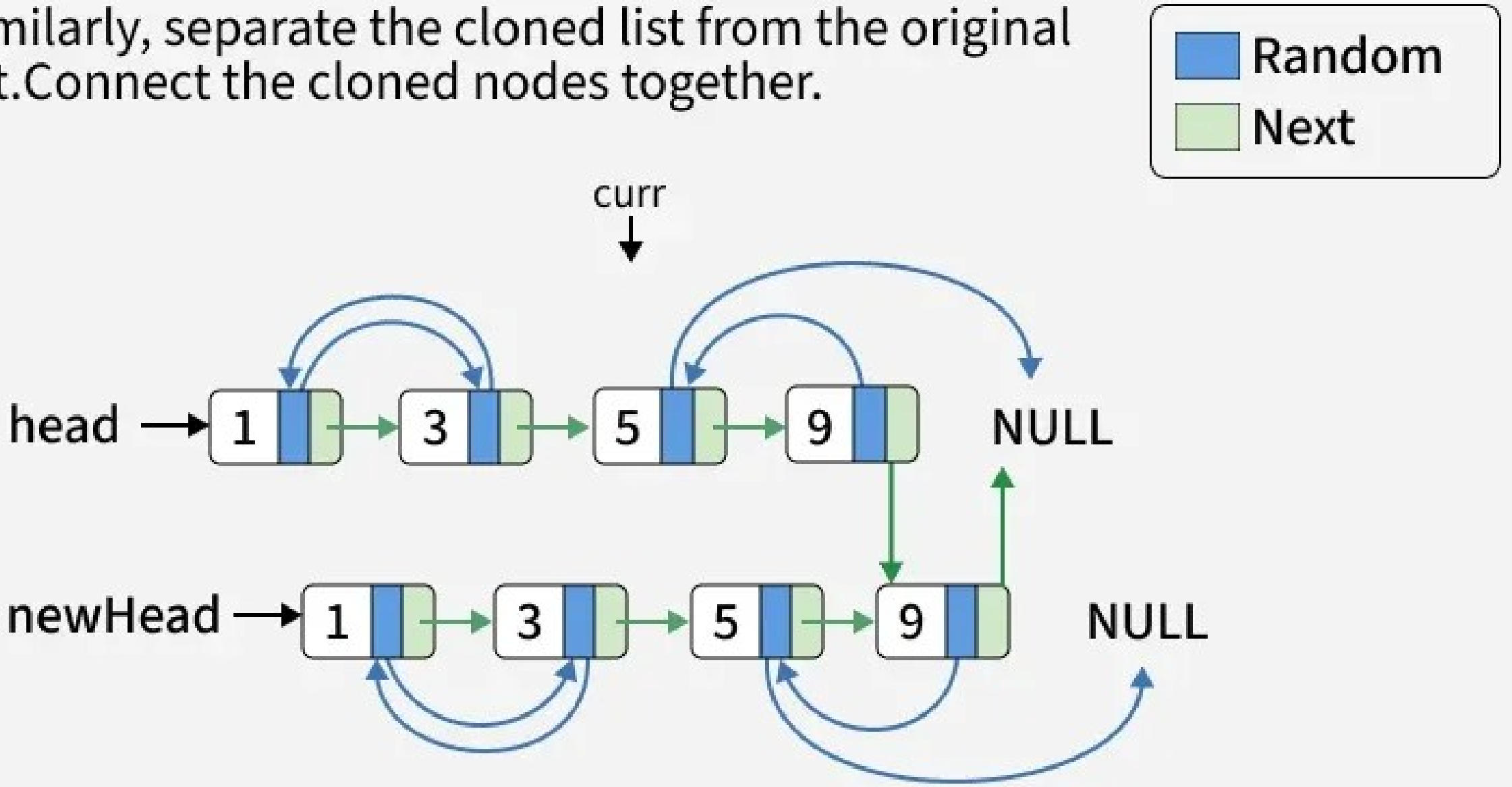
**10** Step

Similarly, separate the cloned list from the original list.Connect the cloned nodes together.



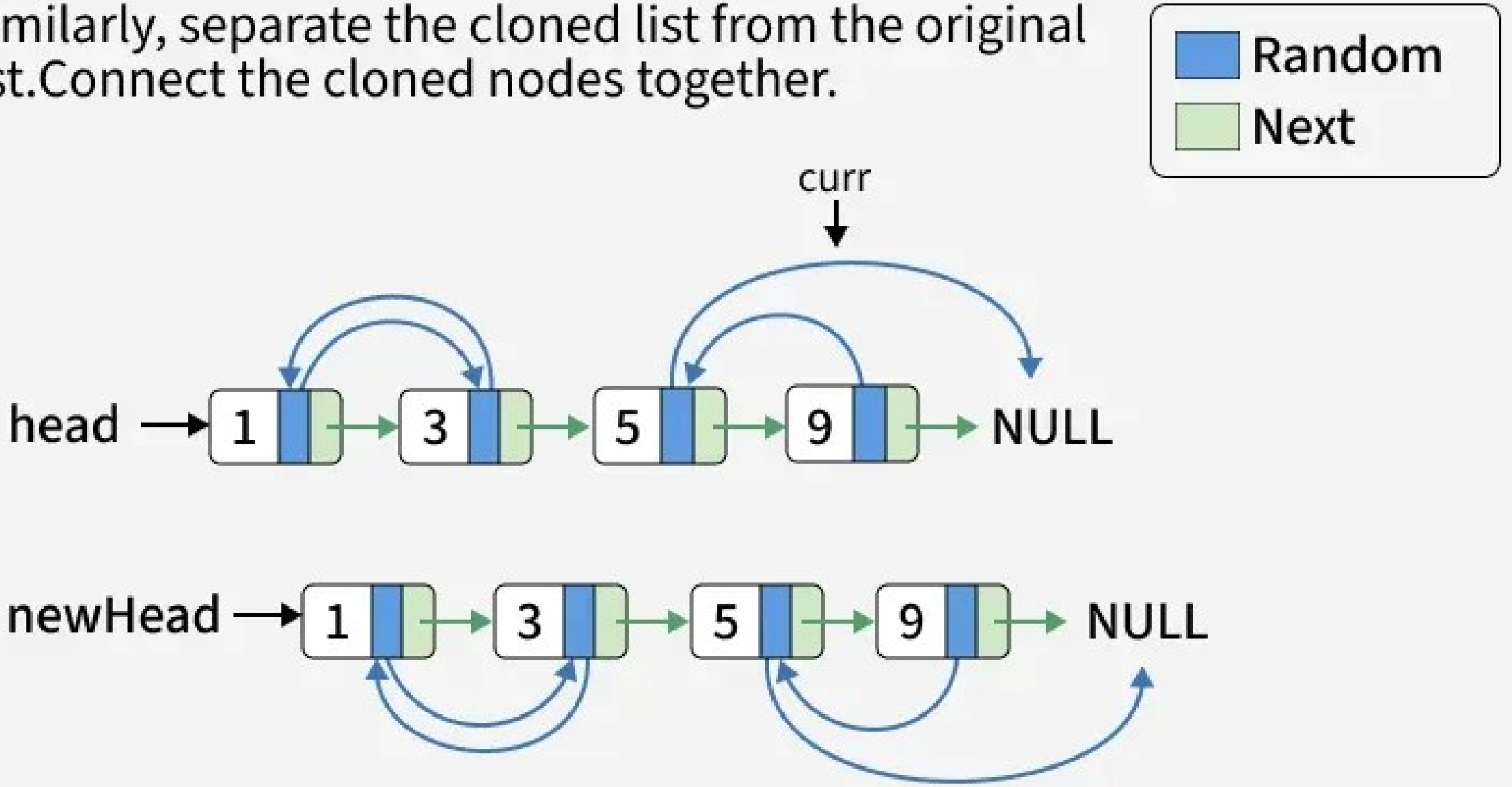
**11** Step

Similarly, separate the cloned list from the original list.Connect the cloned nodes together.



**12** Step

Similarly, separate the cloned list from the original list.Connect the cloned nodes together.



# Group Activity Time!

# Design a hybrid sorting algorithm